



Politechnika Wrocławska

**Prkатыka Programowania,
Zajęcia laboratoryjne,
15:15-16:55,**

Mikołaj Kos

Nazwa:

Most (Bridge)

Problem:

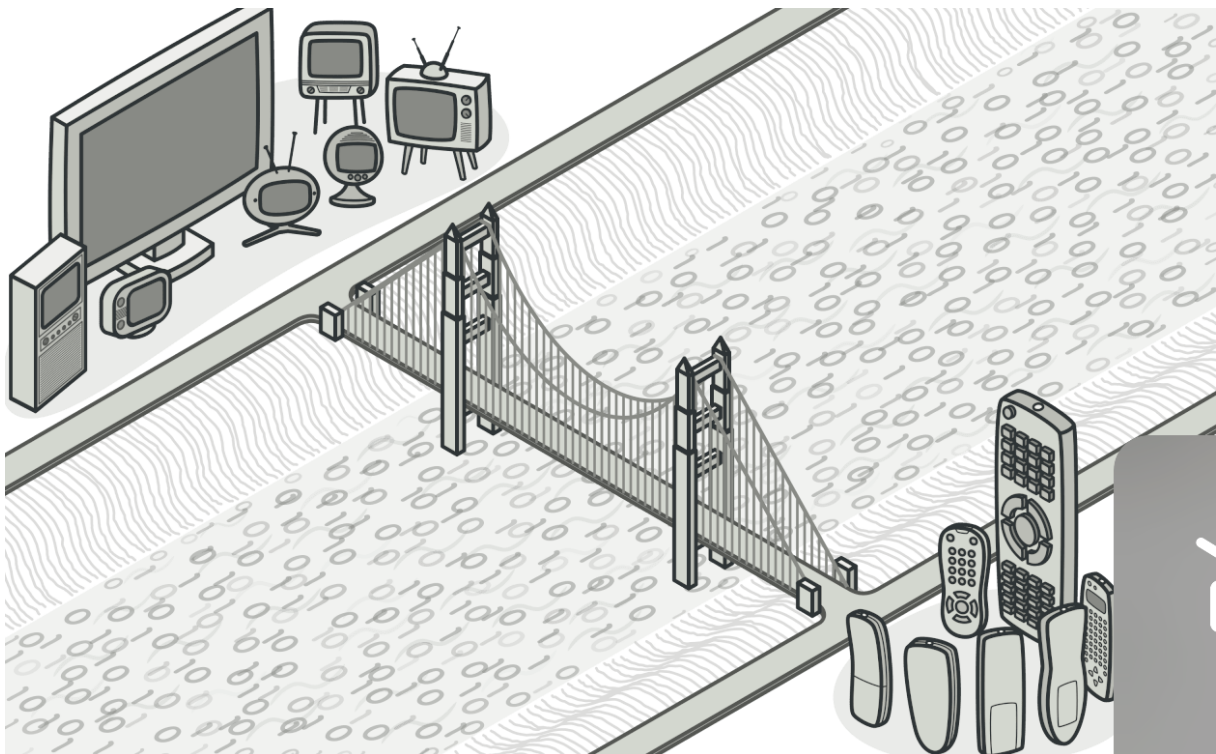
Wzorzec projektowy „Most” rozwiązuje problem niekontrolowanego wzrostu liczby klas spowodowanego wieloma kombinacjami wariantów implementacji i abstrakcji. Celem wzorca jest oddzielenie abstrakcji od jej implementacji, tak aby mogły rozwijać się niezależnie.

Typowe sytuacje, w których warto zastosować wzorzec Most:

- chcesz oddzielić abstrakcję (np. UI) od jej implementacji (np. API),
- system ma wiele wymiarów zmian, np. różne typy urządzeń i różne sposoby ich obsługi,
- dodanie nowych wariantów w obu wymiarach powinno być możliwe bez zmieniania istniejącego kodu.

Warunki wstępne zastosowania:

- istnieją dwie niezależne hierarchie klas (np. „kształty” i „kolory”),
- chcesz uniknąć tworzenia dużej liczby klas dla każdej kombinacji tych hierarchii,



Rozwiązanie

Wzorzec Most dzieli klasę na dwie oddzielne hierarchie:

- Abstrakcję - interfejs reprezentujący funkcjonalność z punktu widzenia użytkownika,
- Implementację - interfejs zawierający szczegóły działania, który może być różny dla różnych platform, technologii lub wariantów.

Wskazówki implementacyjne:

- Abstrakcja i implementacja mogą być definiowane w oddzielnych bibliotekach, a nawet ładowane dynamicznie.
- Implementacja może być zmieniana w czasie działania.
- Często „Most” jest stosowany w połączeniu z innymi wzorcami, np. Factory (do tworzenia implementacji).

Przykład:

Wyobraź sobie, że tworzysz system do rysowania kształtów, takich jak Koło i Kwadrat, w różnych stylach graficznych, np. wektorowym i pikselowym. Pierwszym pomysłem może być stworzenie klas:

KołoWektorowe,	KwadratWektorowy
KołoPikselowe	KwadratPikselowy

Ale gdy dodasz kolejny styl (np. 3D), liczba klas wzrasta wykładniczo, więc każdy kształt w każdej wersji stylu to nowa klasa. Powstaje tzw. eksplozja klas, która utrudnia rozwój i utrzymanie systemu.

Zamiast tworzyć nowe klasy dla każdej kombinacji, oddzielamy dwa niezależne aspekty:

- Figury (kształty): Koło, Kwadrat, itp.
- Sposób rysowania: Wektorowy, Pikselowy, itp.

Tworzymy dwie osobne hierarchie:

- Figura jako abstrakcyjna klasa z polem rysownik
- Rysownik jako klasa bazowa ze stylami: RysownikWektorowy, RysownikPikselowy

Dzięki temu każda figura deleguje rysowanie do obiektu Rysownik, co tworzy „most” między figurą a stylem rysowania. Teraz możemy dodawać nowe kształty i style niezależnie od siebie.

Konsekwencje

Zalety:

- Umożliwia niezależny rozwój abstrakcji i implementacji.
- Redukuje liczbę klas potrzebnych do obsługi różnych kombinacji cech.
- Sprzyja zasadzie pojedynczej odpowiedzialności.
- Ułatwia rozszerzalność systemu w wielu wymiarach zmian.
- Umożliwia tworzenie klas i aplikacji niezależnych od platformy.
- Upraszcza kod dzięki pracy na wyższym poziomie abstrakcji.
- Izoluje klienta od szczegółów implementacyjnych.

Wady:

- Kod może stać się bardziej skomplikowany, gdy wzorzec zostanie użyty w przypadku zwartej, prostej klasy.
- Wprowadza dodatkowy poziom abstrakcji, co może utrudniać zrozumienie architektury, zwłaszcza na początku.
- Może być nadmiernie złożony w prostych przypadkach, gdzie wystarczyłaby jedna konkretna implementacja.